Maestria: Inteligencia artificial aplicada (MNA)

Materia: Visión computacional para imágenes y video

Profesor Titular: PhD. Gilberto Ochoa Ruiz.

Actividad: 9.4 Google Colab con algoritmo Otsu.

Equipo: 5

Integrantes:

- A01561497 - Denisse María Ramírez Colmenero
- A01793672 - Freddy Armendariz Herrera
- A01793668 - Samuel Elias Flores Gonzalez
- A01793675 - Ernesto Nicanor Santillán Guerrero
- A01793052 - Juan Carlos López Cruz

6. Otsu Thresholding

Table of Contents

1. Libraries
2. Single Thresholding
3. Multi Thresholding

Thresholding is used to create a binary image from a grayscale image

Importing Libraries

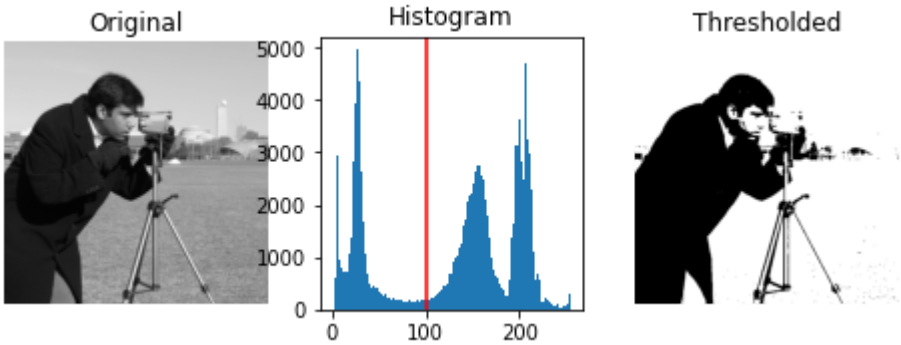
In [ ]:
```
import matplotlib.pyplot as plt
from skimage import data
from skimage.filters import threshold_otsu
from skimage.filters import threshold_multiotsu
import numpy as np
import cv2
```

Single Thresholding

We illustrate how to apply one of these thresholding algorithms. Otsu's method [2]_ calculates an "optimal" threshold (marked by a red line in the histogram below) by maximizing the variance between two classes of pixels, which are separated by the threshold. Equivalently, this threshold minimizes the intra-class variance.

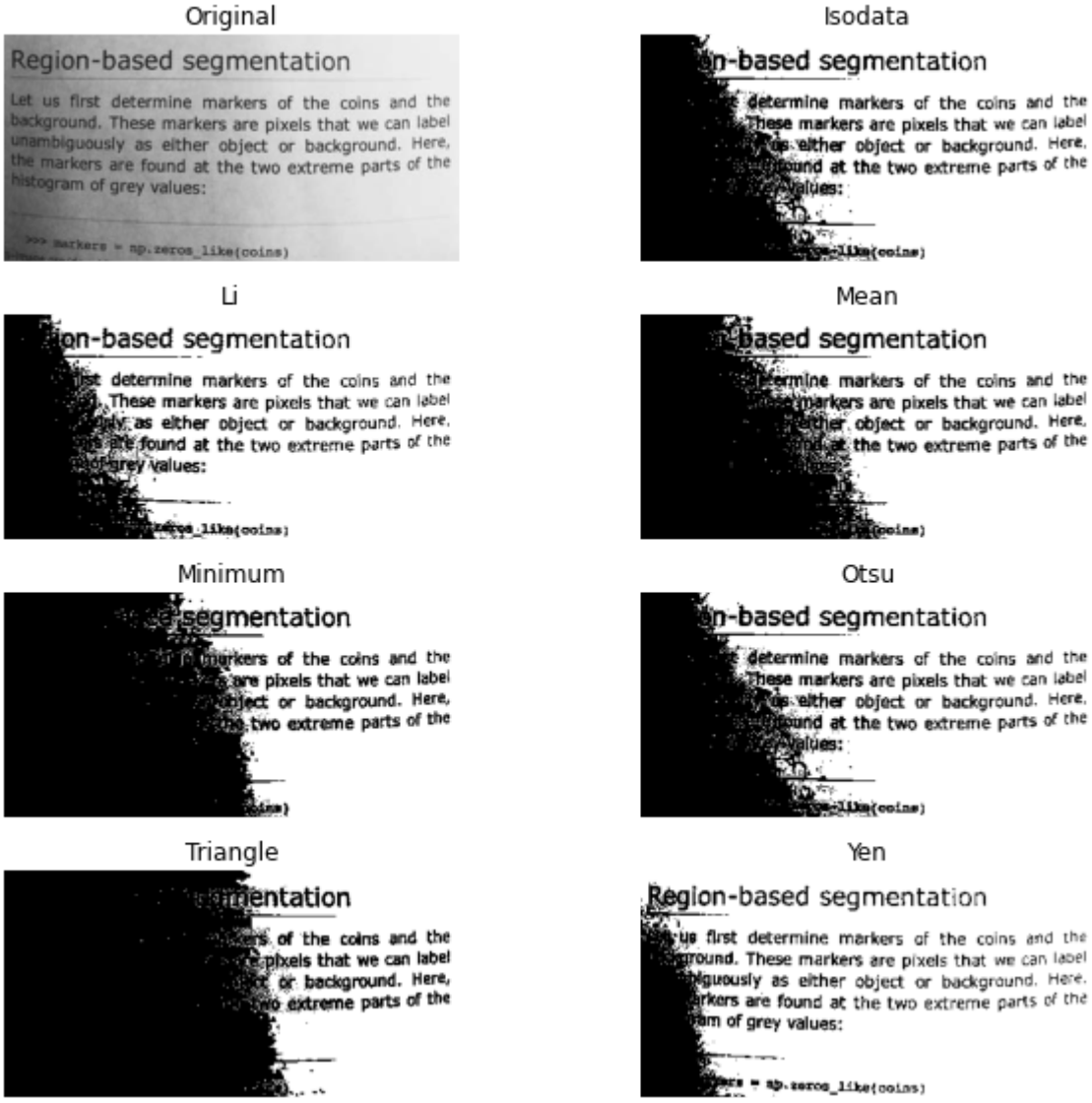
In [ ]:
```
image = data.camera()
thresh = threshold_otsu(image)
binary = image > thresh

fig, axes = plt.subplots(ncols=3, figsize=(8, 2.5))
ax = axes.ravel()
ax[0] = plt.subplot(1, 3, 1)
ax[1] = plt.subplot(1, 3, 2)
ax[2] = plt.subplot(1, 3, 3, sharex=ax[0], sharey=ax[0])

ax[0].imshow(image, cmap=plt.cm.gray)
ax[0].set_title('Original')
ax[0].axis('off')

ax[1].hist(image.ravel(), bins=256)
ax[1].set_title('Histogram')
ax[1].axvline(thresh, color='r')

ax[2].imshow(binary, cmap=plt.cm.gray)
ax[2].set_title('Thresholded')
ax[2].axis('off')

plt.show()
```

```
c:\ProgramData\Anaconda3\envs\PhD\lib\site-packages\skimage\io\manage_plugins.py:23:
UserWarning: Your installed pillow version is < 8.1.2. Several security issues (CVE-2
021-27921, CVE-2021-25290, CVE-2021-25291, CVE-2021-25293, and more) have been fixed
in pillow 8.1.2 or higher. We recommend to upgrade this library.
  from .collection import imread_collection_wrapper
```



If you are not familiar with the details of the different algorithms and the underlying assumptions, it is often difficult to know which algorithm will give the best results. Therefore, Scikit-image includes a function to evaluate thresholding algorithms provided by the library. At a glance, you can select the best algorithm for your data without a deep understanding of their mechanisms.

In [ ]:
```
from skimage.filters import try_all_threshold

img = data.page()

fig, ax = try_all_threshold(img, figsize=(10, 8), verbose=False)
plt.show()
```



In [ ]:
```
# TODO: Slide image for segmentation, alpha blerding
```

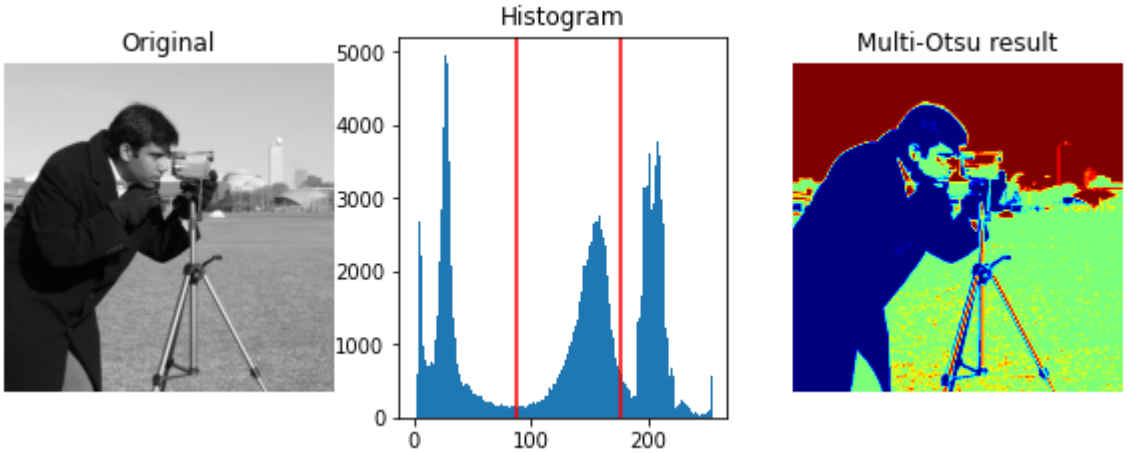
Multi Thresholding

The multi-Otsu threshold is a thresholding algorithm that is used to separate the pixels of an input image into several different classes, each one obtained according to the intensity of the gray levels within the image.

Multi-Otsu calculates several thresholds, determined by the number of desired classes. The default number of classes is 3: for obtaining three classes, the algorithm returns two threshold values. They are represented by a red line in the histogram below.

In [ ]:
```
# The input image.
image = data.camera()

# Applying multi-Otsu threshold for the default value, generating
# three classes.
thresholds = threshold_multiotsu(image)
```

```
# Using the threshold values, we generate the three regions.
regions = np.digitize(image, bins=thresholds)

fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(10, 3.5))

# Plotting the original image.
ax[0].imshow(image, cmap='gray')
ax[0].set_title('Original')
ax[0].axis('off')

# Plotting the histogram and the two thresholds obtained from
# multi-Otsu.
ax[1].hist(image.ravel(), bins=255)
ax[1].set_title('Histogram')
for thresh in thresholds:
    ax[1].axvline(thresh, color='r')

# Plotting the Multi Otsu result.
ax[2].imshow(regions, cmap='jet')
ax[2].set_title('Multi-Otsu result')
ax[2].axis('off')

plt.subplots_adjust()

plt.show()
```



Ejercicios

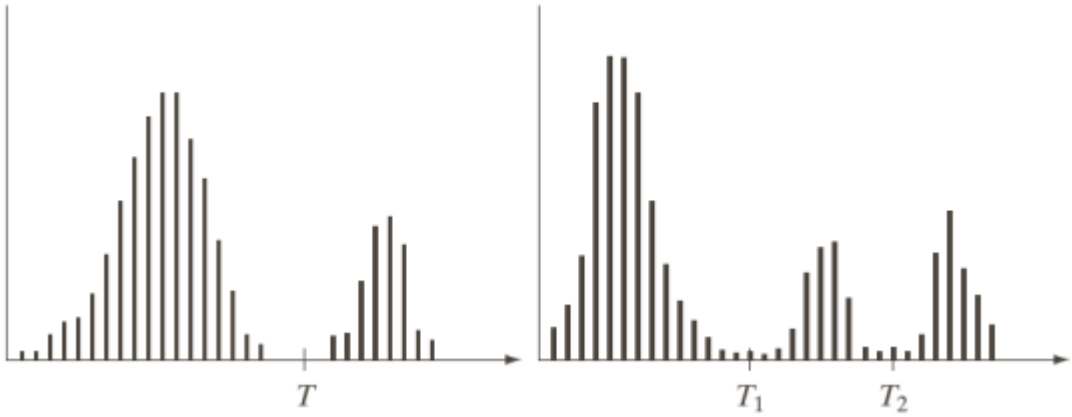
Ejercicio 1

Experimenta con diferentes imagenes ademas de las anteriores. Identifica imagenes con diferentes backgrounds y estilos. Cuales son las limitaciones del *single thresholding* contra el algoritmo de Otsu.

Umbralización o thresholding

Un método básico para diferenciar un objeto del fondo de la imagen es mediante binarización.
• A través del histograma obtenemos una gráfica donde se muestran el número de píxeles por

cada nivel de gris que aparece en la imagen.



Umbralización global: método de Otsu

-El método de Otsu, llamado así en honor a Nobuyuki Otsu que lo inventó en 1979, utiliza técnicas estadísticas, para resolver el problema. En concreto, se utiliza la varianza, que es una medida de la dispersión de valores (en este caso se trata de la dispersión de los niveles de gris).
-Objetivo: calcular el valor umbral de forma que la dispersión dentro de cada clase sea lo más pequeña posible, pero al mismo tiempo la dispersión sea lo más alta posible entre clases diferentes.

Pasos:

- Paso 1: Computar el histograma normalizado de la imagen.
- Paso 2: Computar las sumas acumulativas P1(k), k=0,1,..., L-1.
- Paso 3: Computar las medias acumulativas m(k), k=0,1,..., L-1.
- Paso 4: Computar la media global mG.
- Paso 5: Computar la varianza entre clases
- Paso 6: Maximizar la varianza anterior
- Paso 7: Obtener la medida de separabilidad

El algoritmo de Otsu es una técnica de umbralización automática qiue se utiliza para encontrar automáticamente el valor óptimo de umbral en una imagen. Este algoritmo se base en la idea de maximizar la varianza entre dos clases de píxeles: los que se clasifican como objeto y los que se clasifican como "fondo".

El algoritmo de Otsu calcula la varianza para todos los posibles umbrales y selecciona aquel que maximiza la varianza entre las dos clases de píxeles. Esto permite obtener un umbral que separa de manera óptima las regiones de interés de la imagen.

El simple thresholding es una técnica básica de procesamiento de imágenes que se utulizar para transformar una imagen en escala de grises a una imagen binaria, donde los píxeles se clasifican en dos categorías: negro o blanco.

En esta técnica, se elige un valor de umbral (threshold) y se compara con los valores de intensidad de los píxeles de la imagen. Si el valor de intensidad de un píxel es mayor que el valor del umbral, el píxel se convierte en blanco, de lo contrario se convierte en negro.

Por otro lado, la umbralización múltiple permite dividir una imagen en varias clases utilizando más de un valor de umbral. Esto es útil cuando se desea segmentar una imagen en más de dos regiones. Por ejemplo, se puede utilizar la umbralización múltiple para segmentar una imagen de una célula en tres regiones diferentes: el núcleo, la membrana y el citoplasma.

En resumen, la umbralización simple es útil cuando se desea dividir una imagen en dos regiones (como es el caso de las imagenes con texto), mientras que la umbralización múltiple es más adecuada para la segmentación de imágenes en múltiples regiones.

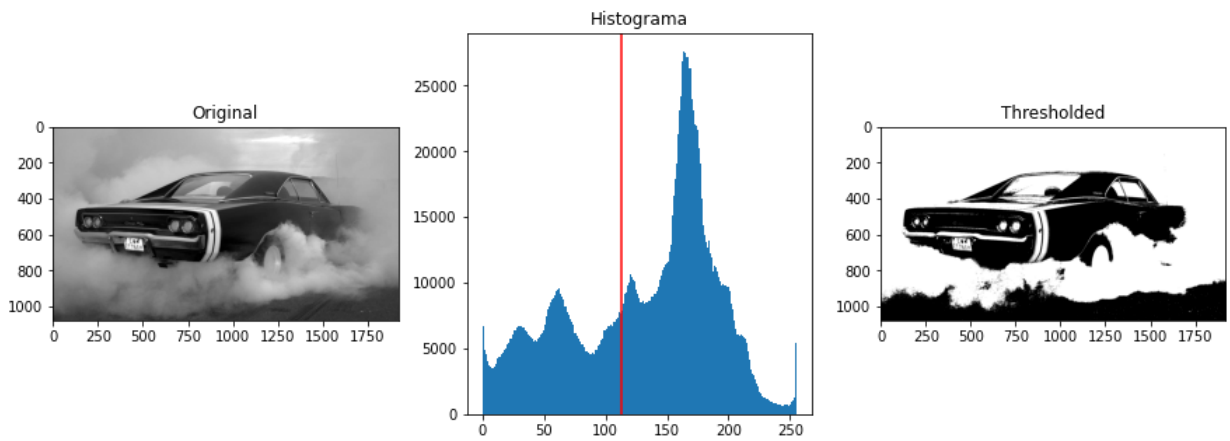
In [ ]:
```
def UmbralSimple(picture):
  # Leer imagen y convertir a escala de grises
  image = cv2.imread(picture,0)
  # Definicion del umbral
  thresh = threshold_otsu(image)
  # Aplicacion del umbral a la imagen
  binary = image > thresh
  # Declaracion de grafico
  fig = plt.figure(figsize=(15,5))
  # Desplegar imagen original
  a = fig.add_subplot(1,3,1)
  imgplot = plt.imshow(image, cmap=plt.cm.gray)
  a.set_title("Original")
  # Desplegar histograma
  a = fig.add_subplot(1,3,2)
  imgplot = plt.hist(image.ravel(), bins=256)
  a.set_title("Histograma")
  a.axvline(thresh, color='r')
  # Desplegar imagen binarizada
  a = fig.add_subplot(1,3,3)
  imgplot = plt.imshow(binary, cmap=plt.cm.gray)
  a.set_title("Thresholded")
  # Desplegar grafico completo
  plt.show()
```

In [ ]:
```
def UmbralMultiple(picture):
  # Leer imagen y convertir a escala de grises
  image = cv2.imread(picture,0)
  # Definicion de los umbrales
  thresholds = threshold_multiotsu(image)
  # Aplicacion del umbral a la imagen
  regions = np.digitize(image, bins=thresholds)
  # Declaracion de grafico
  fig = plt.figure(figsize=(15,5))
  # Desplegar imagen original
  a = fig.add_subplot(1,3,1)
  imgplot = plt.imshow(image, cmap="gray")
  a.set_title("Original")
  # Desplegar histograma
  a = fig.add_subplot(1,3,2)
  imgplot = plt.hist(image.ravel(), bins=256)
```

```
  a.set_title("Histograma")
  for thresh in thresholds:
    a.axvline(thresh, color='r')
  # Desplegar imagen binarizada
  a = fig.add_subplot(1,3,3)
  imgplot = plt.imshow(regions, cmap="jet")
  a.set_title("MultiOtsu")
  # Desplegar grafico completo
  plt.show()
```

Ejemplo 1

In [ ]:
```
pic = "charger.jpg"
UmbralSimple(pic)
```



In [ ]:
```
pic = "charger.jpg"
UmbralMultiple(pic)
```

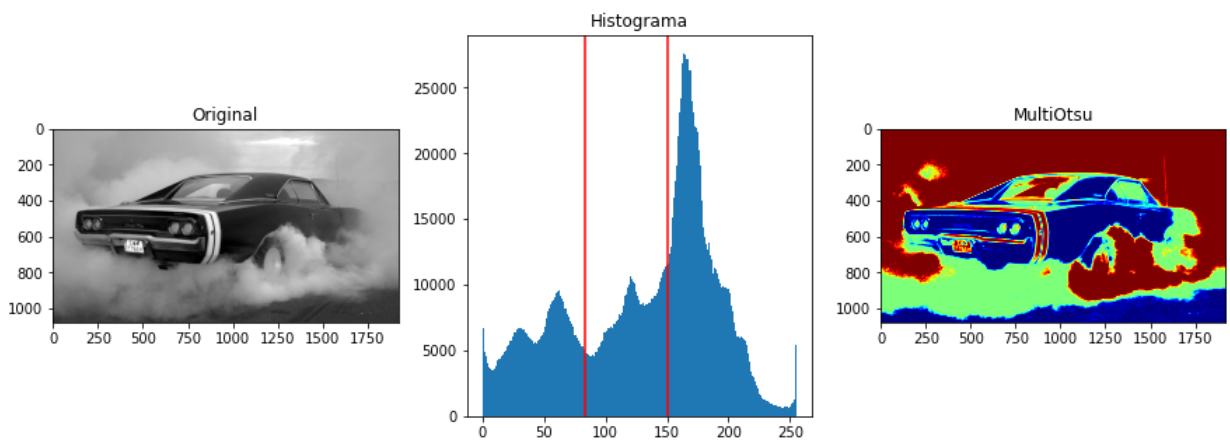
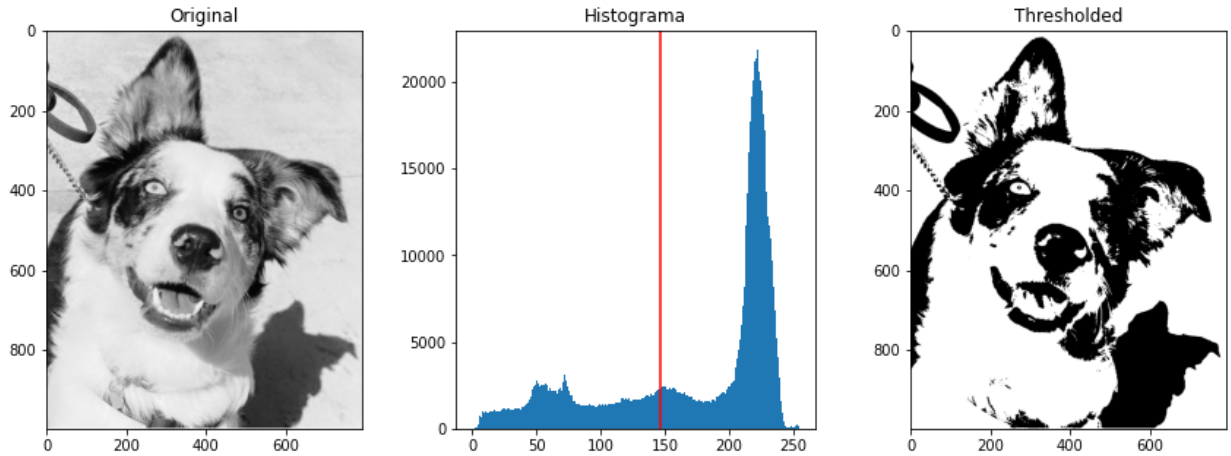


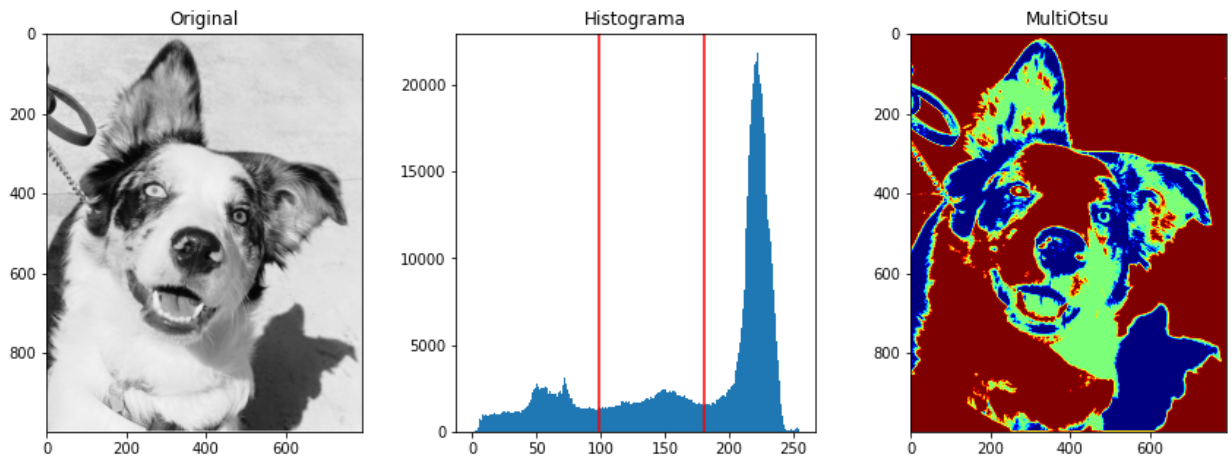
Imagen de un objeto se observa que la segmentación usando single es buena pero al ser un imagen con un objeto obscuro parte del fondo no se segmenta como lo es el piso, este efecto sucede similar al aplicar multi thresholding ya que lo divide en 3 clases pero en este caso se ve que disminuye la cantidad del fondo (el piso) en comparación con el objeto.

Ejemplo 2

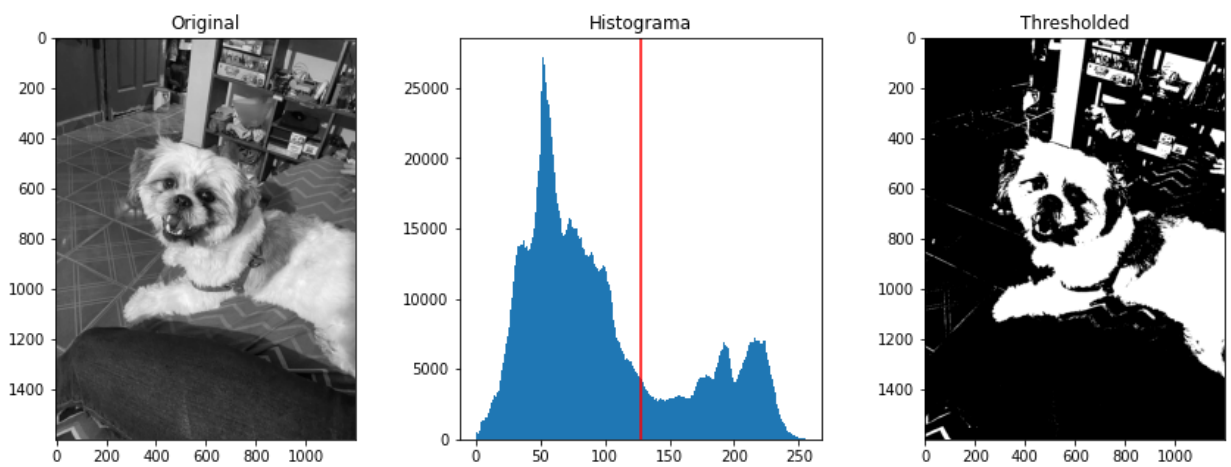
In [ ]:
```
pic = "Mera.jpg"
UmbralSimple(pic)
```



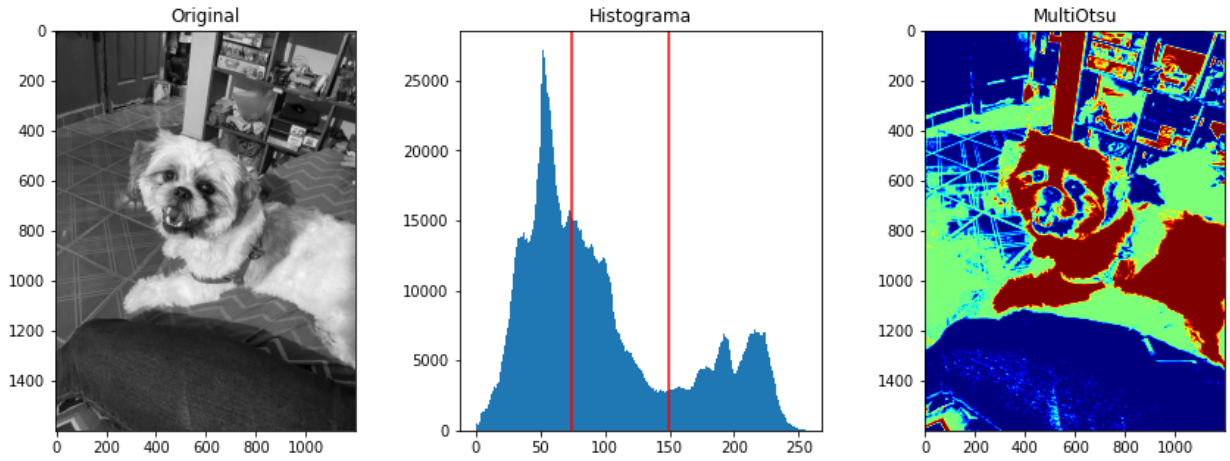
In [ ]:
```
pic = "Mera.jpg"
UmbralMultiple(pic)
```



In [ ]:
```
pic = "Perrito.jpeg"
UmbralSimple(pic)
```



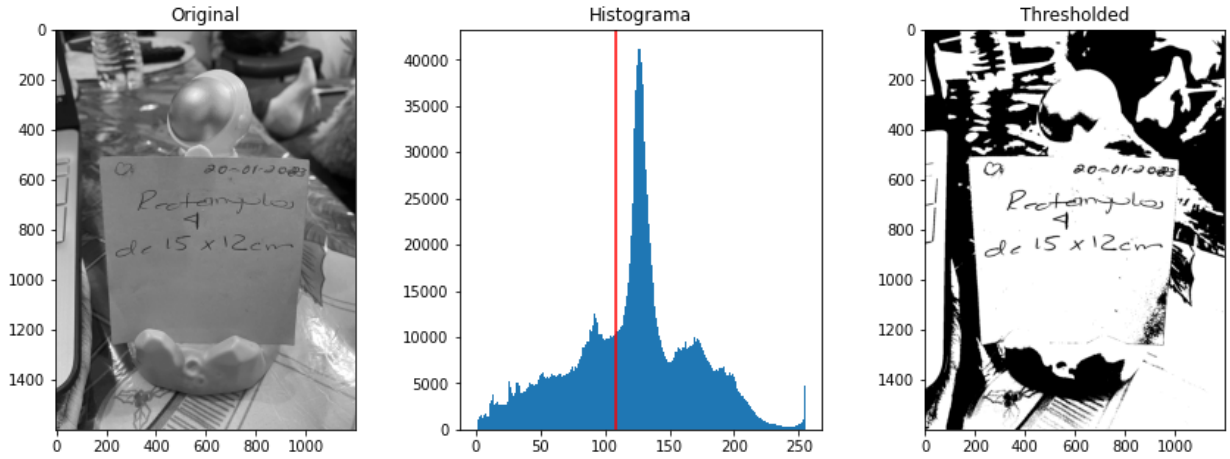
In [ ]:
```
pic = "Perrito.jpeg"
UmbralMultiple(pic)
```



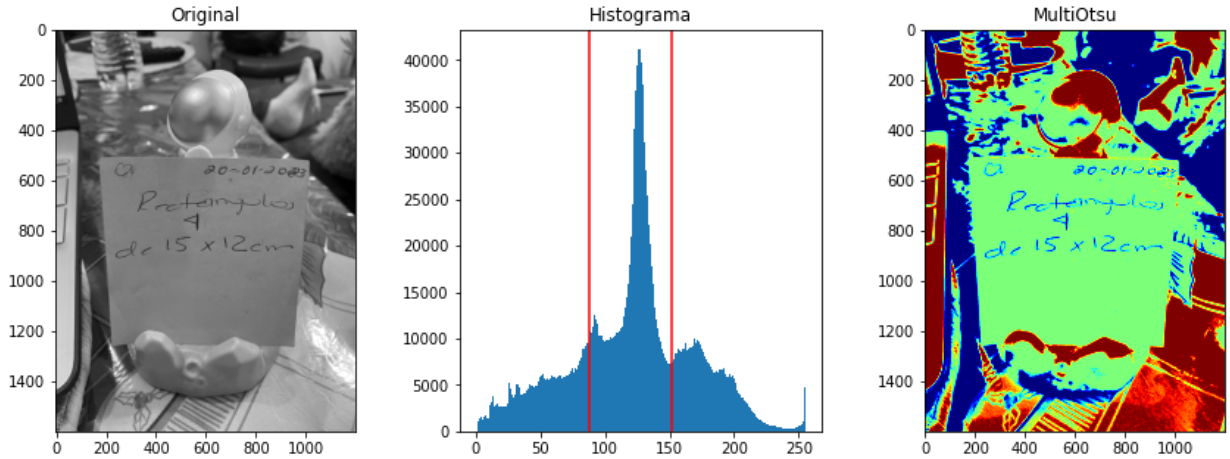
Se observa que las sombras al ser del mismo tono que el perrito observado es muy difícil de segmentar por aparte ya que en ambos algoritmos single y multi la cantidad de sombra lo sigue filtrando con los colores obscuros del perrito.

Ejemplo 4

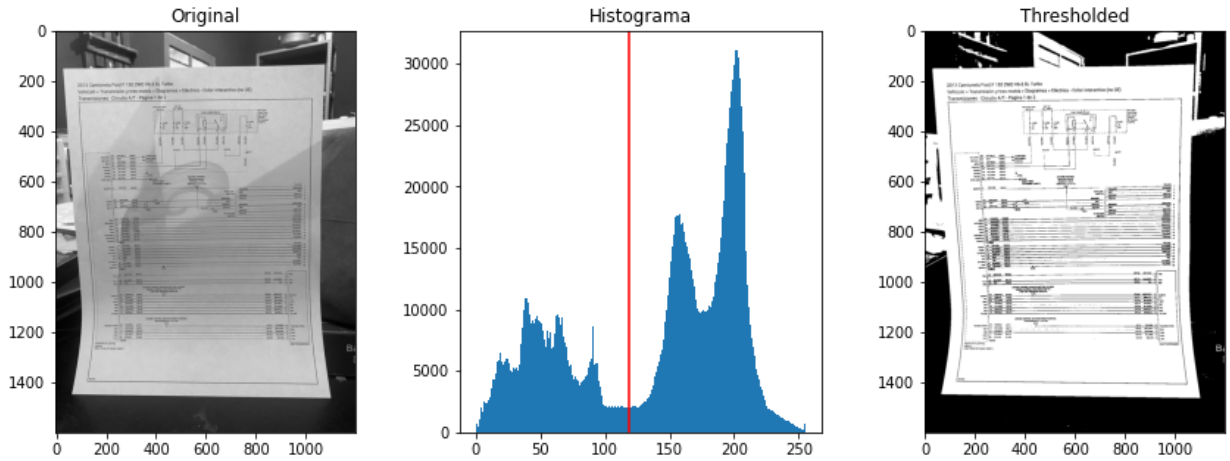
In [ ]:
```
pic = "Astro.jpeg"
UmbralSimple(pic)
```



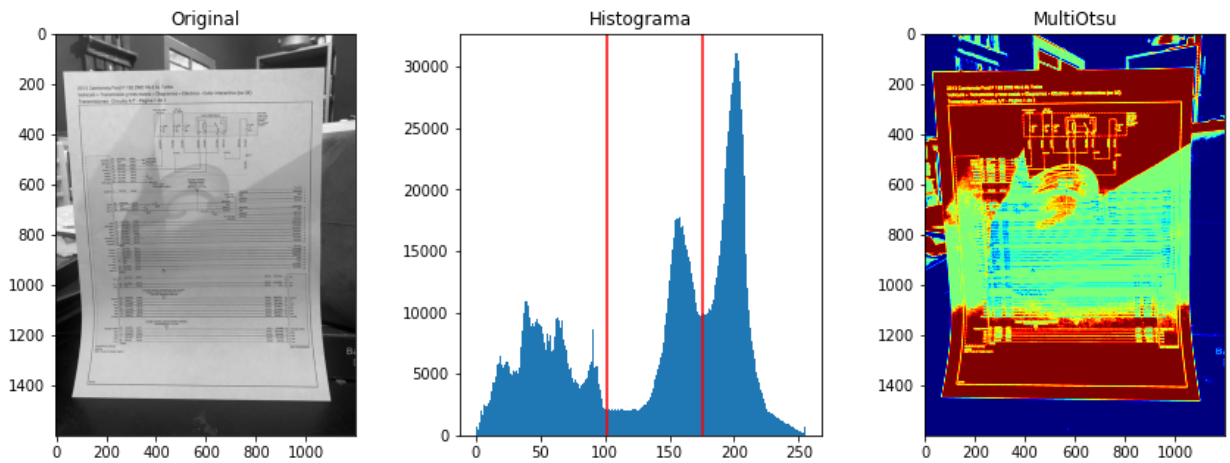
In [ ]:
```
pic = "Astro.jpeg"
UmbralMultiple(pic)
```



In [ ]:
```
pic = "Diagram.jpeg"
UmbralSimple(pic)
```



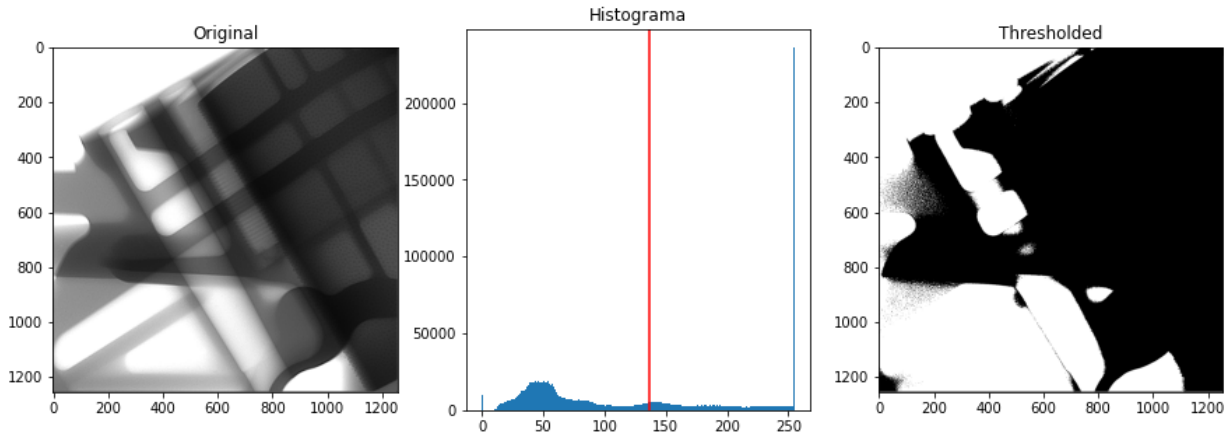
In [ ]:
```
pic = "Diagram.jpeg"
UmbralMultiple(pic)
```



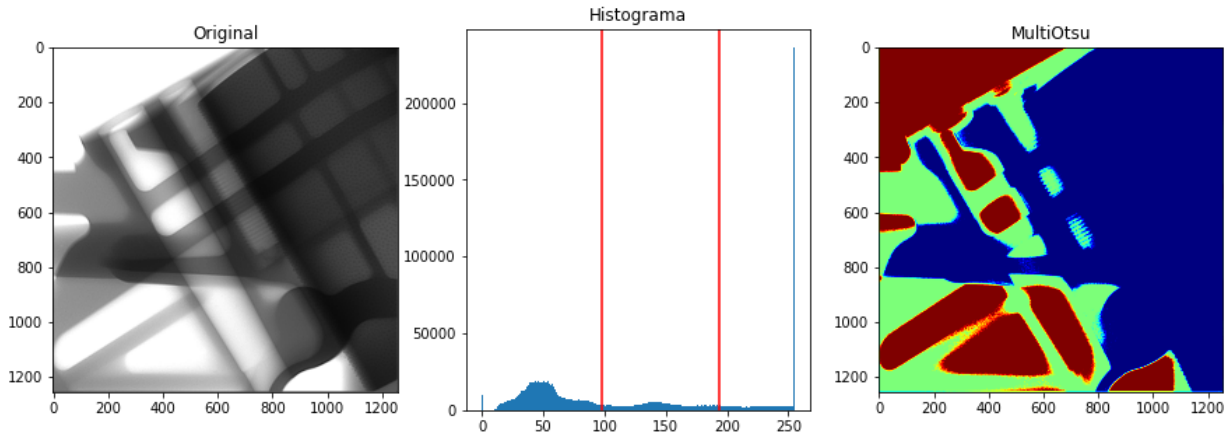
Se aprecia como en ambas imagenes existe texto, y este se aclara o contrasta mas al utilizar un threshold simple ya que se binariza quedando mas notable el texto al quedar en color negro y dejar todo el fondo en color blanco.

Ejemplo 4

In [ ]:
```
pic = "Xray.png"
UmbralSimple(pic)
```



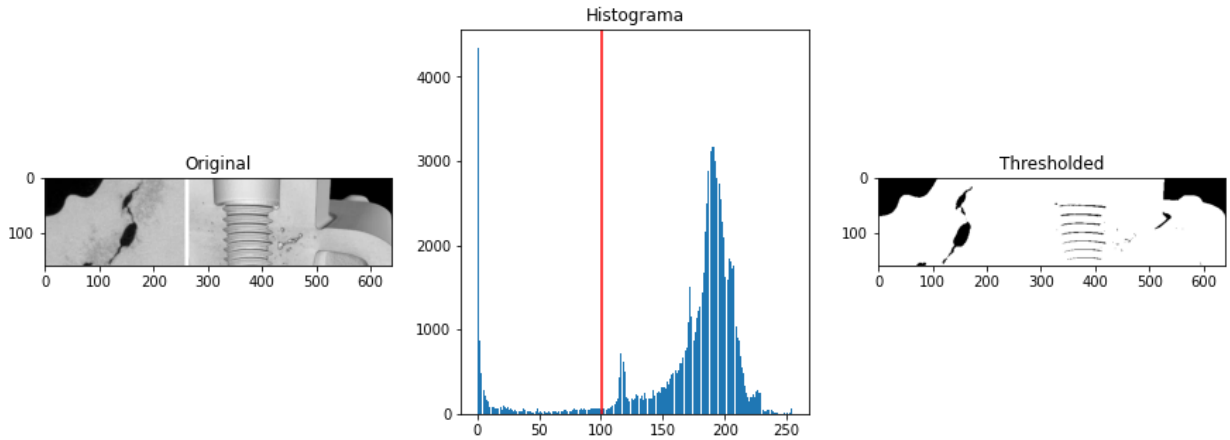
In [ ]:
```
pic = "Xray.png"
UmbralMultiple(pic)
```



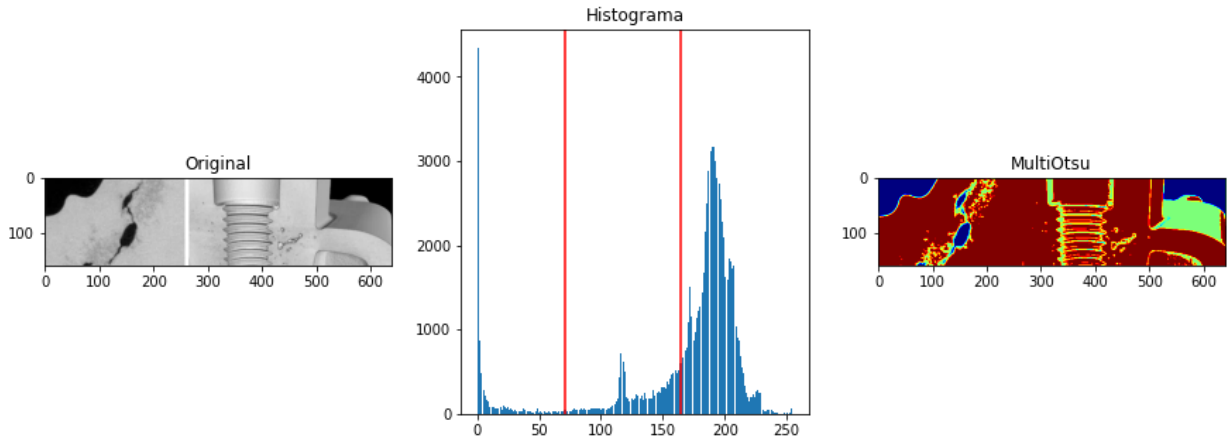
Rayos X de una pieza de casting se observa que al parecer multi si ofrece una ventaja sobre single ya que permite ver mejor la división de los contrastes y en su defecto podría permitir identificar mejor los defectos como porosidades o grietas.

Ejemplo 5

In [ ]:
```
pic = "shrinkage.jpg"
UmbralSimple(pic)
```



In [ ]:
```
pic = "shrinkage.jpg"
UmbralMultiple(pic)
```



Análisis de imperfecciones se observa que single oculta ciertos defectos que en multi se observan mejor como lo es la grieta y la porosidad junto a la cuerda.

Ejercicio 2

Asi como en un proyecto previo, es posible aplicar Otsu para ventanas de diferentes tamaños, obteniendo mejores resultados. Realiza esta implementacion y ve como mejoran los resultados con el ejemplo de la hoja de papel.

Instalación de libreria image_slicer

In [ ]:
```
!pip install image_slicer
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/p
ublic/simple/
Requirement already satisfied: image_slicer in /usr/local/lib/python3.9/dist-packages
(2.1.1)
Requirement already satisfied: Pillow==7.2.0 in /usr/local/lib/python3.9/dist-package
s (from image_slicer) (7.2.0)
```

Librerias

In [ ]:
```
from PIL import Image
from PIL import Image
import image_slicer
from numpy import asarray
from skimage import exposure
```

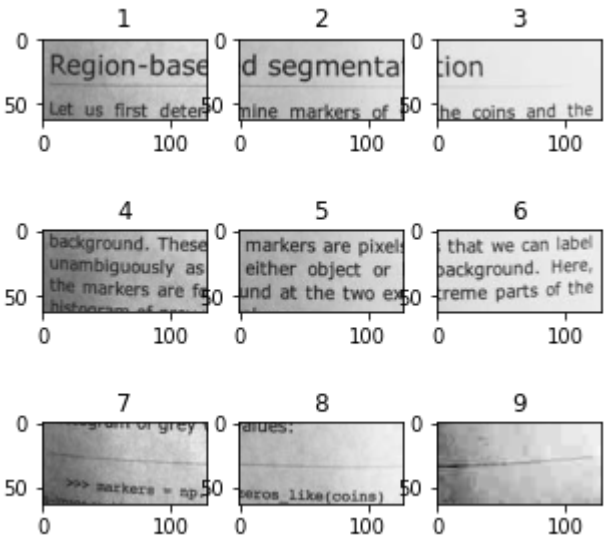
Ejemplo de otsu mostrando la división de ventanas en cada paso.

Usaremos 9 ventanas para la demostración.

In [ ]:
```
img = data.page()
im = Image.fromarray(img)# Conversion de imagen page de array a objeto image
im.save("/content/page.jpg") # Se guarda la imagen por que image_slicer no lee arrays.
```

```
#Division de la imagen en mosaicos
# Aplicación de otsu single a cada mosaico obtenido
n=9;
print("Division de la imagen en: %s ventanas",n)
mosaicos=[]
grises=[]
fig = plt.figure(figsize=(5, 5))
tiles = image_slicer.slice('/content/page.jpg', n, save=False)
for k in range(0,len(tiles)):
  if np.sqrt(n)% np.round(np.sqrt(n))==0:
    a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))),k+1)
  else :
    a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))+1),k+1)
  mosaicos.append(asarray(tiles[k].image))
  plt.imshow(mosaicos[k],cmap='gray')
  a.set_title(str(k+1))
```

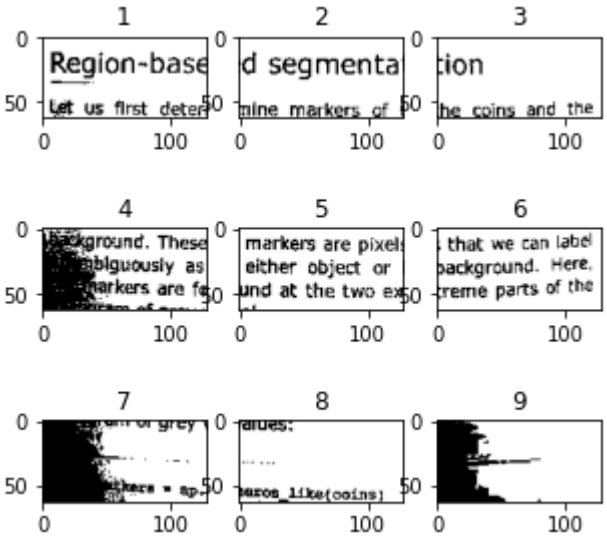
```
Division de la imagen en: %s ventanas 9
```



In [ ]:

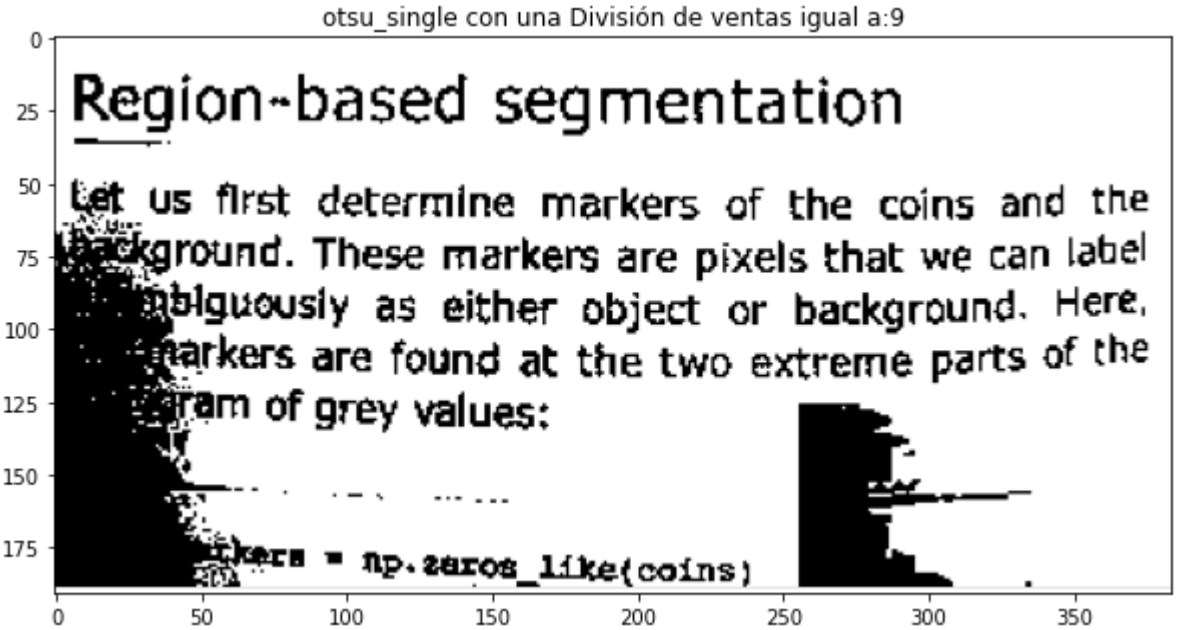
```
# Aplicación de otsu single a cada mosaico obtenido
print("Aplicación de otsu single a cada mosaico obtenido ")
tiles_otsu_bin=[]
n=len(mosaicos)
fig = plt.figure(figsize=(5, 5))
for k in range(0,len(mosaicos)):
   if np.sqrt(n)% np.round(np.sqrt(n))==0:
     a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))),k+1)
   else :
     a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))+1),k+1)
   thresh = threshold_otsu(mosaicos[k])
   binary = mosaicos[k] > thresh
   tiles_otsu_bin.append(binary)
   tiles[k].image=Image.fromarray(tiles_otsu_bin[k])
   plt.imshow(tiles[k].image,cmap=plt.cm.gray)
   a.set_title(str(k+1))
```

```
Aplicación de otsu single a cada mosaico obtenido
```



In [ ]:
```
largo,ancho =img.shape
img_otsu=image_slicer.main.join(tiles, width=ancho, height=largo)
fig = plt.figure(figsize=(10, 8))
plt.title("otsu_single con una División de ventas igual a:"+str(len(tiles)))
plt.imshow(img_otsu)
```

Out[ ]:
```
<matplotlib.image.AxesImage at 0x7f372ae5d1f0>
```



Creación de funciones para facilitar el Análisis.

Función para dividir la imagen en mosaicos

In [ ]:
```
def mosaicos (img,n): # numero de slides debe ser en 2 4 6 9 12 16 20 25 30
  im = Image.fromarray(img)# Conversion de imagen page de array a objeto image
  im.save("/content/page.jpg") # Se guarda la imagen por que image_slicer no lee array
  mosaicos=[]
  grises=[]
  #fig = plt.figure(figsize=(10, 10))
  tiles = image_slicer.slice('/content/page.jpg', n, save=False)
```

```
  for k in range(0,len(tiles)):
    if np.sqrt(n)% np.round(np.sqrt(n))==0:
      a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))),k+1)
    else :
      a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))+1),k+1)
    mosaicos.append(asarray(tiles[k].image))
    #plt.imshow(mosaicos[k],cmap='gray')
    #a.set_title(str(k+1))
  return mosaicos, tiles
```

Función para aplicar Otsu single a cada mosaico

In [ ]:
```
def mosaicos_otsu(mosaicos, tiles):
  tiles_otsu_bin=[]
  n=len(mosaicos)
  #fig = plt.figure(figsize=(10, 10))
  for k in range(0,len(mosaicos)):
    if np.sqrt(n)% np.round(np.sqrt(n))==0:
      a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))),k+1)
    else :
      a=fig.add_subplot(int(np.round(np.sqrt(n))),int(np.round(np.sqrt(n))+1),k+1)
    thresh = threshold_otsu(mosaicos[k])
    binary = mosaicos[k] > thresh
    tiles_otsu_bin.append(binary)
    tiles[k].image=Image.fromarray(tiles_otsu_bin[k])
    #plt.imshow(tiles[k].image,cmap=plt.cm.gray)
    #a.set_title(str(k+1))
  return tiles
```

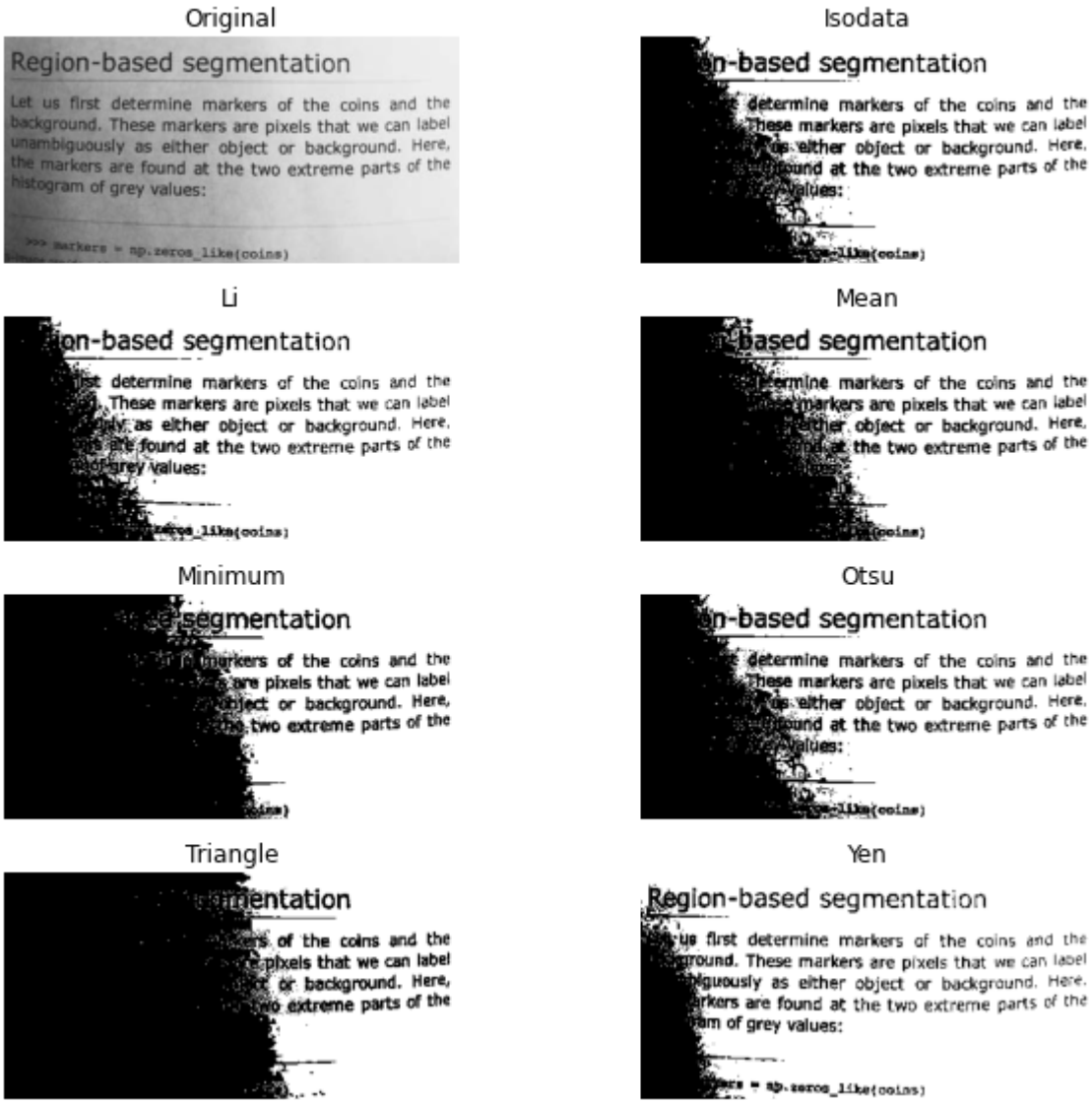
Función para unir cada mosaico despues de la Segmentación Otsu single.

In [ ]:
```
def mosaicos_joint(imag,tiles):
  largo,ancho =imag.shape
  img_otsu=image_slicer.main.join(tiles, width=ancho, height=largo)
  fig = plt.figure(figsize=(10, 8))
  plt.title("otsu_single con una División de ventas igual a:"+str(len(tiles)))
  plt.imshow(img_otsu)
```

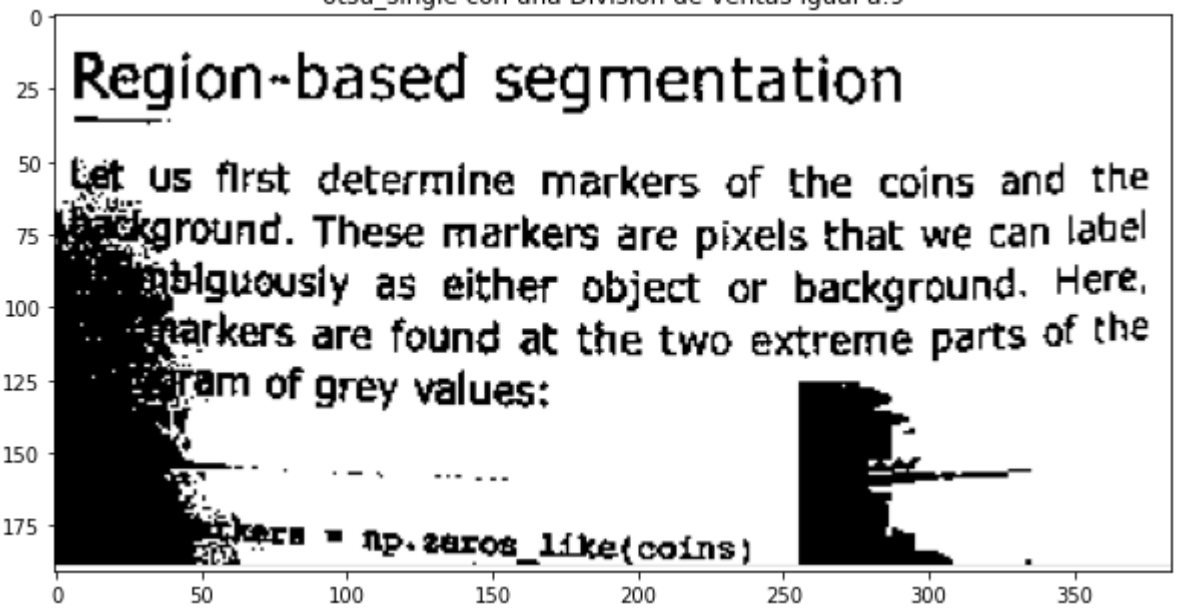
Comparacion de otsu por ventana contra los demas metodos.

In [ ]:
```
fig, ax = try_all_threshold(img, figsize=(10, 8), verbose=False)
plt.show()
img = data.page()
### Otsu con 9 ventanas.
a,b=mosaicos(img,9)
t=mosaicos_otsu(a,b)
mosaicos_joint(img,t)
### Otsu con 20 ventanas.
a,b=mosaicos(img,20)
t=mosaicos_otsu(a,b)
mosaicos_joint(img,t)
### Otsu con 42 ventanas.
a,b=mosaicos(img,42)
t=mosaicos_otsu(a,b)
mosaicos_joint(img,t)
```

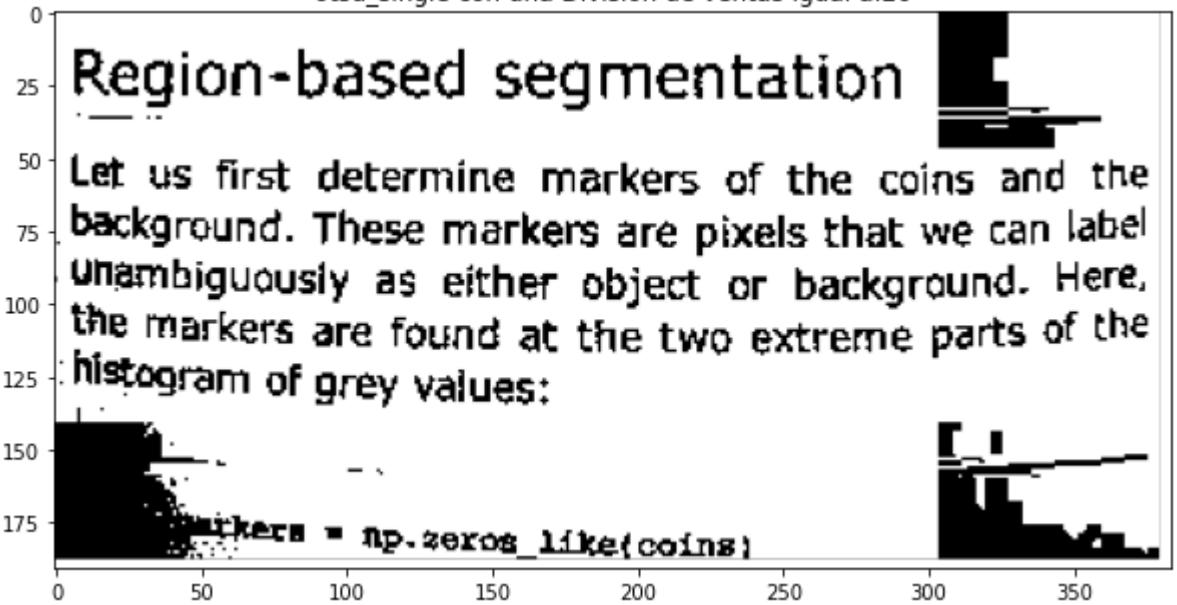
```
### Otsu con 100 ventanas.
a,b=mosaicos(img,100)
t=mosaicos_otsu(a,b)
mosaicos_joint(img,t)
### Otsu con 210 ventanas.
a,b=mosaicos(img,210)
t=mosaicos_otsu(a,b)
mosaicos_joint(img,t)
```



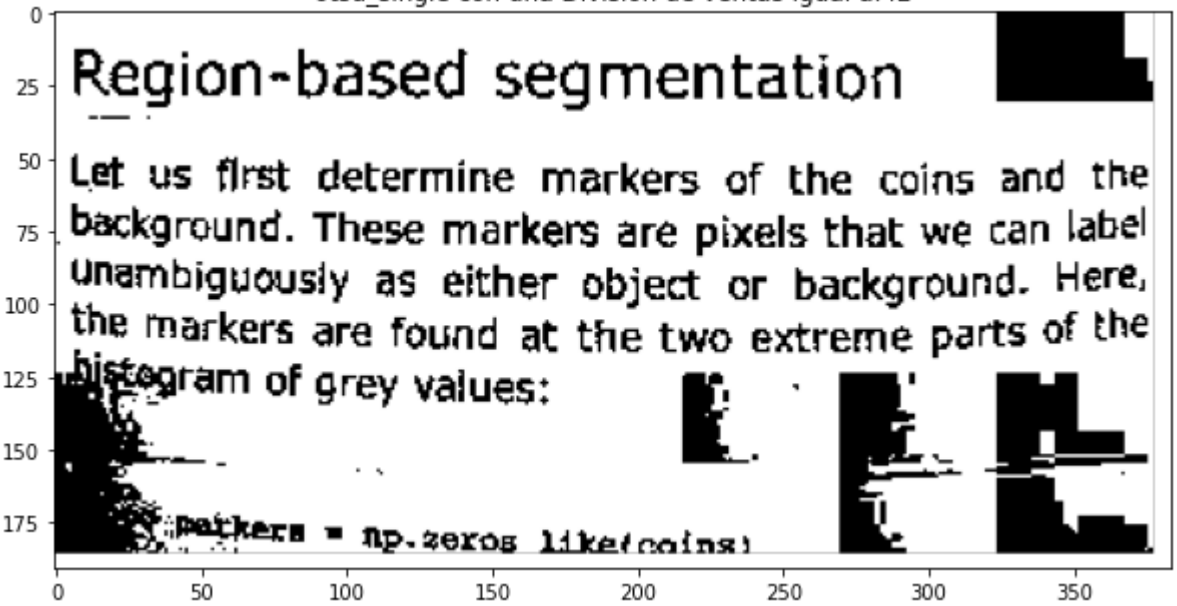
otsu_single con una División de ventas igual a:9



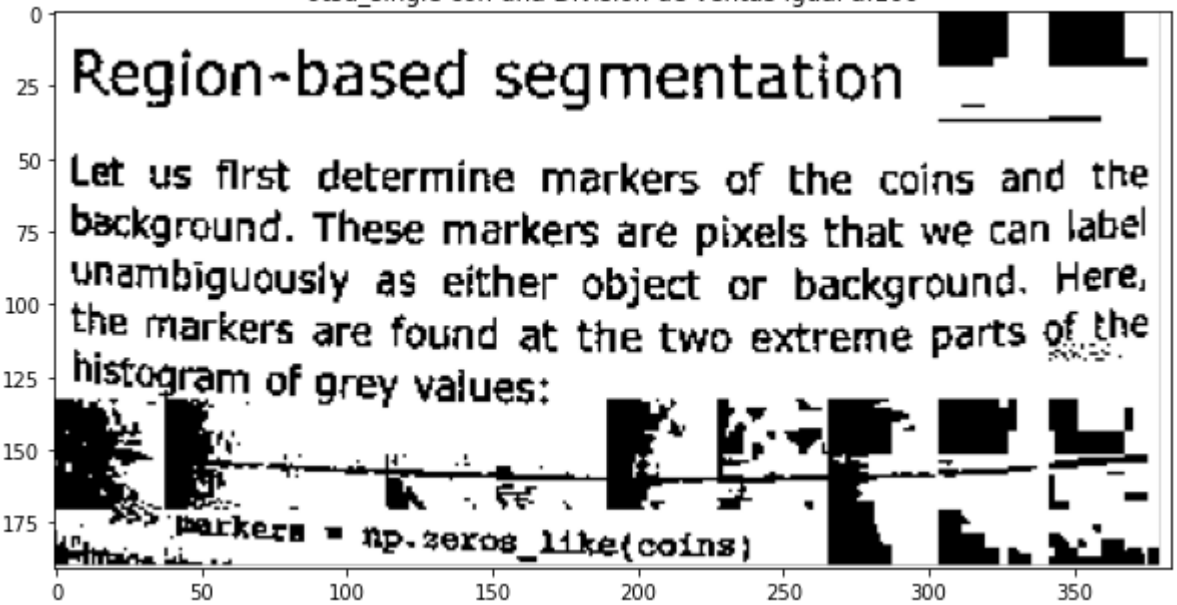
otsu_single con una División de ventas igual a:20

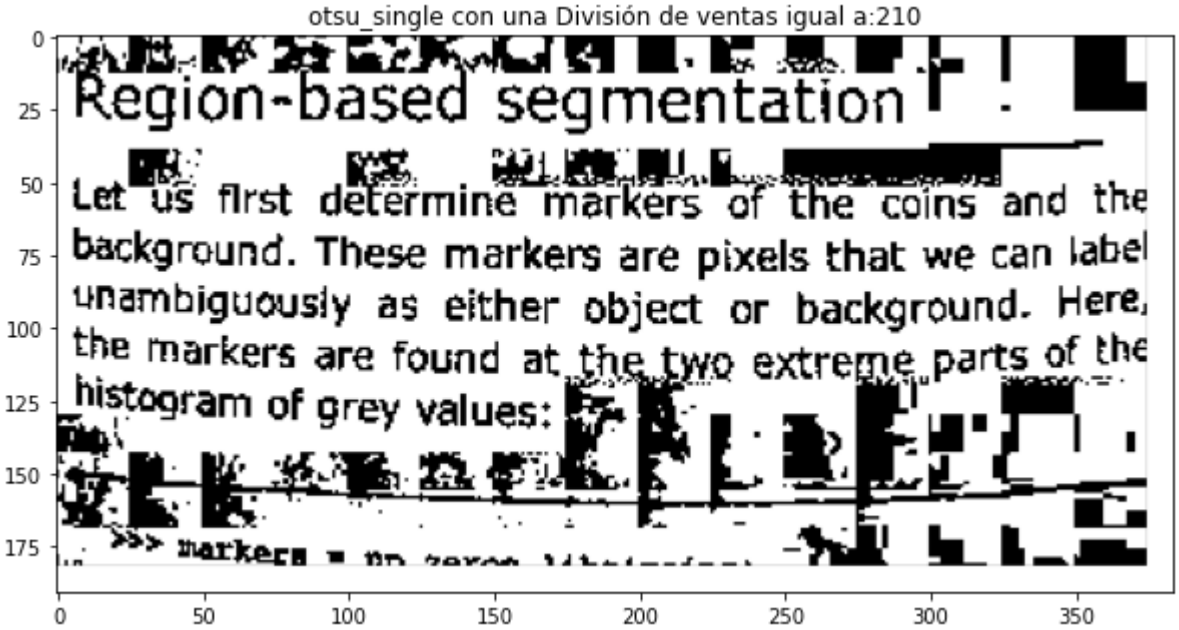


otsu_single con una División de ventas igual a:42



otsu_single con una División de ventas igual a:100





Conclusión:

- Analizando los 7 métodos de segmentación por thresholding parace que para este ejemplo de la hoja de papel con texto el mejor es **Yen** para manejar el efecto de reflectancia que se ve en la imagen original. *Otsu no ofrece un mal resultado, pero se pierde aproximadamente un 20% de la información de la hoja de papel.
- Después de comparar la segmentación por thresholding del mejor que fue Yen con la iteración de 9 ventanas usando otsu single disminuyo la cantidad de información perdida en la hoja.
- Pero ojo a medida que la cantidad de ventanas aumenta va aumentando la cantidad de información perdida para este ejemplo se usaron 5 casos, 9 ventanas, 20 ventanas, 42 ventanas, 100 ventanas y 210 ventanas. *La que arrojo un resultado mejor fue 20 ventanas, apartir de 42 ventanas se empieza a ver como va aumentando la cantidad de cuadros negros y en 210 se ve como se pierde el texto inferior.*La cantidad de cuadros o ventanas que se dividirá la imagen va a depender de la imagen y el objeto que necesitemos segmentar.

Referencias

- Multilevel image thresholds using Otsuâ s method - MATLAB multithresh. (s. f.). https://www.mathworks.com/help/images/ref/multithresh.html
- Thresholding – Image Processing with Python. (s. f.). Data Carpentry - Image Processing with Python. https://datacarpentry.org/image-processing/07-thresholding/
- Thresholding — Introduction to Bioimage Analysis. (s. f.). https://bioimagebook.github.io/chapters/2-processing/3-thresholding/thresholding.html
- Wikipedia contributors. (2023, 19 febrero). Thresholding (image processing). Wikipedia. https://en.wikipedia.org/wiki/Thresholding_(image_processing)
- R. Gonzalez & R. Woods. (2018). Digital Image Processing.Pearson.

- OpenCV.(s.f). Image Thresholding. https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- Mimi. (2021).Umbralización openCV python.kipunaEc. https://noemioocc.github.io/posts/Umbralizaci%C3%B3n-openCV-python/